10월 5주차 sync

지난주에 한 것

- DiffKV 최적화: DiffKV-v1
  - attention과 engine의 불필요한 device 동기화 제거
  - 32개 layer가 각각 GPU에서 읽던 KV cache 길이를 한 번에 CPU로 복사하도록 변경
- DiffKV 프로파일링 과정 및 결과 정리
  - 실험 간 latency 편차와 FP16, DiffKV-v1 사이의 비정상적인 GEMM 시간 차이 조사
  - thermal throttling을 확인하고, base clock 고정과 온도 관리 후 재측정
  - FP16과 DiffKV가 공통 실행 가능한 14개 workload의 FP16, DiffKV, DiffKV-v1을 nsys로 kernel latency breakdown

이번주에 한 것

- Attention 최적화
  - Nsight Compute (ncu)로 Decode Attention kernel 병목 찾고 최적화
  - 결과: 기존 DiffKV Attention 대비 최대 **2.5x** speedup
- MATH benchmark 도입
  - synthetic workload와 GSM8K는 FP16 KV Cache memory가 차지 않아 DiffKV의 이점이 없는 workload
    - preemption이 발생하지 않으므로…
  - Correctness를 위해 DeepSeek-R1-Distill-Llama-8B 포팅
    - 해당 모델을 구 vLLM target commit이 지원하고 있지 않으므로 (`vLLM 0.2.6`) llama3 RoPE 부분 패치
  - correctness test 완료

목차

0. 실험 조건
1. 기존 구현의 문제점
   1.1. DiffKV Attention의 비효율성
   1.2. Attention 구현 비교
   1.3. Attention 병목
   1.4. 결론
2. DiffKV-v2: decode attention 최적화
   2.1. 변경
   2.2. Optimization Results
3. MATH benchmark
   3.1. workload의 이점과 기존 benchmark의 한계
   3.2. DeepSeek-R1-Distill-Llama-8B 포팅
   3.3. Correctness 측정 방법과 결과
   3.4. E2E 측정 방법
   3.5. 주의사항
4. 다음 주 계획
참고 자료

0. 실험 조건

항목	내용
GPU	RTX 3090 24 GB x2, SM clock 1395 MHz 고정, 전력 제한 350 W
Workload	• Synthetic data: Meta-Llama-3-8B • MATH: DeepSeek-R1-Distill-Llama-8B
FP16 (Baseline)	• vLLM 0.2.6 (commit `1db83e3`): full FP16 • 해당 vLLM 버전에서는 DeepSeek-R1-Distill-Llama-8B를 지원하지 않으므로 llama3 RoPE 패치를 적용하여 사용할 수 있도록 함
DiffKV	• DiffKV: original DiffKV • DiffKV-v1: DiffKV보다 조금 더 최적화된 놈 • DiffKV-v2: DiffKV-v1보다 조금 더 최적화된 놈

1. 기존 구현의 문제점

1.1. DiffKV Attention의 비효율성

	E2E (s)	GEMM	Attention
FP16	**73.57**	**47.92 (65.1%)**	**13.22 (18.0%)**
DiffKV	105.81	48.03 (45.4%)	18.44 (17.4%)
DiffKV-v1	77.06	48.03 (62.3%)	18.43 (23.9%)

(B, ISL, OSL) = (8, 2048, 2048) synthetic data 기준 Attention이 FP16 대비 DiffKV가 30%가량 느렸고, 다른 케이스에서도 일관되게 30-40% 정도의 성능 하락이 있었습니다. 이 중에서 Decode가 Prefill 대비 실행시간이 훨씬 길기 때문에 Decode Attention에 대해서만 프로파일 후 최적화를 진행했습니다.

1.2. Attention 구현 비교

FP16 prefill은 CUTLASS 구성요소를 사용하는 FlashAttention CUDA C++ 커널, DiffKV prefill은 자체 Triton 커널입니다. decode는 두 경로 모두 자체 CUDA C++ 커널입니다. 여기서 Decode Attention은 아래와 같이 main과 reduce로 나눌 수 있습니다.

- main: Attention 계산 본체, PV 계산 후 output을 temp buffer에 저장
- reduce: 각 CTA가 들고있는 temp buffer를 reduce해 최종 Attention output tensor 생성

Dtype	Pass	Kernel	Implementation
FP16	Prefill	`pytorch_flash::flash_fwd_kernel`	PyTorch FlashAttention2
FP16	Decode main	`vllm::paged_attention_v2_kernel`	vLLM 자체 CUDA Kernel
FP16	Decode reduce	`vllm::paged_attention_v2_reduce_kernel`	vLLM 자체 CUDA Kernel
DiffKV-v1, v2	Prefill	`attn_fwd`	Triton
DiffKV-v1	Decode main	`vllm::sparse_paged_attention_wrapper`	CUDA
DiffKV-v1	Decode reduce	`vllm::reduce_kernel`	CUDA
DiffKV-v2	Decode main	`register_gqa_attention_kernel`	CUDA
DiffKV-v2	Decode reduce	`register_gqa_reduce_kernel`	CUDA

1.3. Attention 병목

- 비효율적인 KV Cache load
  - Key vector 구조체에 정렬 지정이 없어 8 byte tensor를 2 byte load 4번으로 읽고 있었음 (Value vector도 이와 마찬가지의 비효율성이 있음)
- dequantization overhead
  - load한 값을 바로 dequantization해서 쓰므로 memory latency hiding이 안되고 있었음
  - head별 중복 dequantization: KV head 하나를 공유하는 query head 4개가 같은 KV Cache를 각자 load하고 각자 dequantization

decode main, step 1024, layer 15	FP16	DiffKV-v1
DRAM Read / Write	112.6 MB	**33.4 MB**
DRAM throughput	**748.7 GB/s** (peak의 82%)	163.0 GB/s (peak의 18%)
Kernel latency	**150.3 us**	205.0 us
Load Instruction (SASS)	• key `LDG.E.64` (8 byte) • value `LDG.E.128` (16 byte)	• key `LDG.E.U16` (2 byte) • value `LDG.E.U16` (2 byte)
Occupancy	**38.9%**	30.9%

1.4. 결론

논문에 보고된 성능보다 떨어지는 이유는 앞서 설명한 DiffKV decode attention kernel의 비효율에 있고, 그 결과 DRAM bandwidth를 theoretical peak의 18%만 쓰는 latency bound kernel이 되었기 때문입니다.

2. DiffKV-v2: decode attention 최적화

2.1. 변경

2.1.1. Decode main

- KV Cache load align
  - AS-IS: key vector 하나를 담는 `struct k_vec_type`는 `uint16_t data[4]`, 즉 8byte지만 memory alignment는 원소 type 크기인 2 byte이므로, compiler가 2 byte 단위로 read하는 instruction을 만듦. (value vector도 마찬가지)
  - TO-BE
    - `k_vec_type`: `alignas(8)`, `LDG.E.64` SASS Instruction
    - `v_vec_type`: `alignas(4)`, `LDG.E` SASS Instruction (4 byte)
  - 결과: ncu로 `B=8, context_len=3072` profiling 결과 decode main latency가 227us → 189us, global memory load instruction 수가 2.02M → 0.87M로 감소
- Dequantization optimization
  - CTA 병렬화
    - AS-IS: CTA(256 thread) 하나가 query head 하나와 context 전체를 맡음
    - TO-BE: 변경 후 CTA(128 thread) 하나가 KV head 하나와 그 query head 4개를 맡고, context를 일정 token씩 나눠 CTA 수를 늘림
  - Register reuse
  - XOR swizzling: PV partial reduce에서 XOR swizzling 적용
  - 결과: 앞선 동일한 조건에서 decode main latency가 189us → 92.8us, global memory load instruction 수가 0.87M → 0.50M, 전체 instruction 수가 46.5M → 23.7M로 감소

2.1.2. Decode reduce

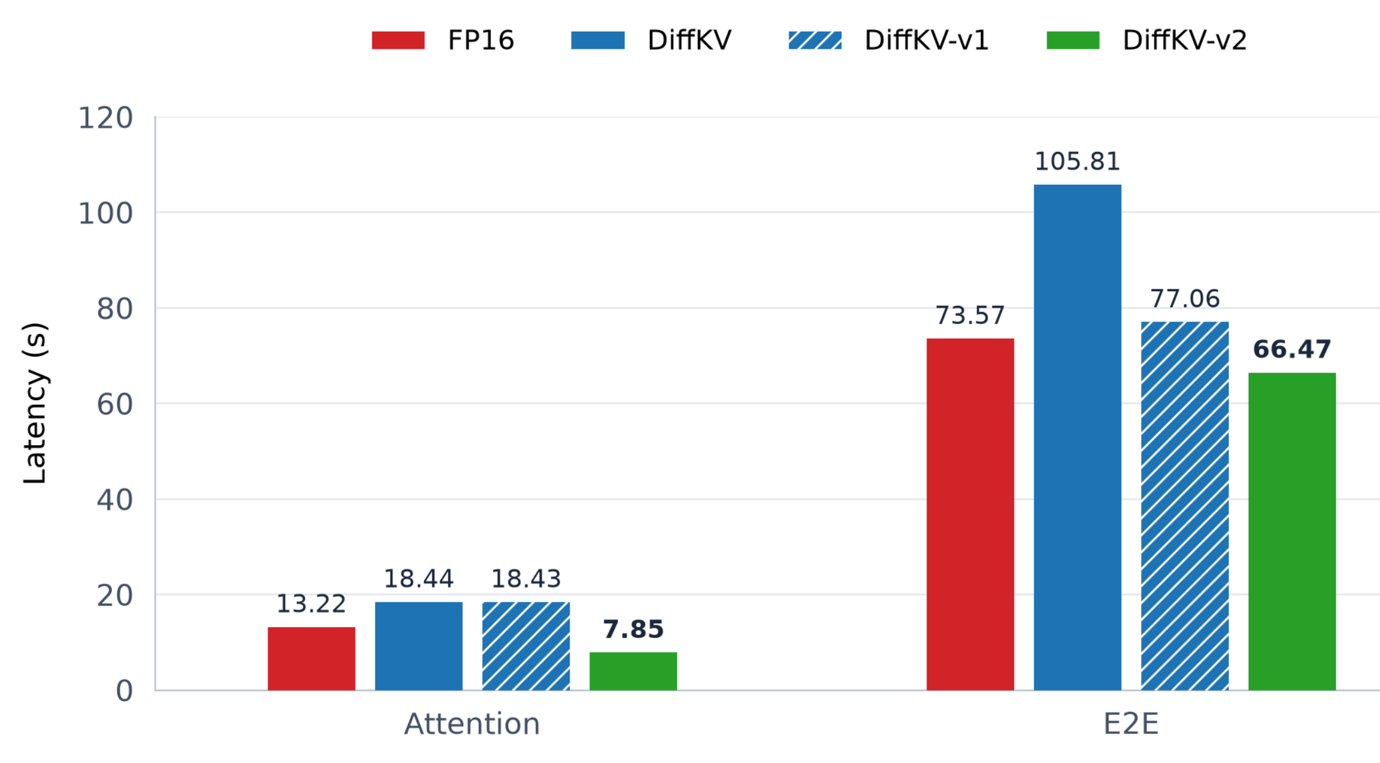
- CTA 병렬화
- Register reuse
- FP64 제거: `(sum + 1e-6) * factor`의 `1e-6`이 double이라 FP64 MAC 연산을 `1e-6f`로 바꿔 제거
- 결과: 29.9 us → 10.5 us로 감소

2.2. Optimization Results

2.2.1. Attention latency

Context	DiffKV (us)	DiffKV-v2	Perf Ratio
512	65.1	**37.4**	1.74
2048	204.8	**82.6**	2.48
3072	282.0	**117.4**	2.40
4096	361.7	**153.1**	2.36
6000	532.0	**206.1**	2.58
8192	691.5	**281.0**	2.46

2.2.2. End-to-end latency breakdown: (B, ISL, OSL) = (8, 2048, 2048)



경로	E2E (s)	GEMM	Attention	KV Update	Other GPU	Others
FP16	73.57	47.92 (65.1%)	13.22 (18.0%)	**0.28 (0.4%)**	**2.08 (2.8%)**	9.97 (13.6%)
DiffKV	105.81	48.03 (45.4%)	18.44 (17.4%)	2.79 (2.6%)	2.25 (2.1%)	34.13 (32.3%)
DiffKV-v1	77.06	48.03 (62.3%)	18.43 (23.9%)	2.79 (3.6%)	2.27 (2.9%)	**5.45 (7.1%)**
DiffKV-v2	**66.47**	48.05 (72.3%)	**7.85 (11.8%)**	2.79 (4.2%)	2.27 (3.4%)	**5.43 (8.2%)**

3. MATH benchmark

3.1. workload의 이점과 기존 benchmark의 한계

workload의 이점: DiffKV를 vLLM에 붙였을 때 FP16보다 빨라지는 workload를 말합니다. 조건은 동시에 도는 요청의 context token 합계가 FP16 KV cache 용량(RTX 3090, 8B 모델, gpu_memory_utilization=0.9에서 38.9K~40.1K token)을 넘는 것입니다. 그러면 FP16은 preemption (recomputation)이 생겨 decode batch가 줄고, DiffKV는 같은 메모리에 약 3배 token을 담아 높은 batch를 유지하여 가속화할 수 있습니다. 이를 위해 long context와 high concurrency가 필요합니다.

- **Synthetic data**: 해당 offline workload에서는 preemption이 없으니 KV Cache Compression을 통한 scheduling 이점이 0이고, 따라서 개별 kernel latency의 효율성이 성능을 좌우한다고 볼 수 있습니다. 기존 synthetic data로 측정한 결과 DiffKV가 FP16대비 성능이 낮아보였던 이유도 여기에 있겠습니다.
- **GSM8K**: prompt 680~843 token, 출력 평균 113 token, batch=8. 최대 context가 8 × (843 + 1024) ≈ 15K token으로 FP16에서 peak memory가 40% 이하라 KV Cache 압박이 없으며, 반복 출력으로 FP16과 DiffKV의 출력 길이가 달라 number 비교도 공평하지 않았던 문제가 있었습니다.

이를 위해 MATH-500 밴치마크를 도입하여 측정했습니다.

3.2. DeepSeek-R1-Distill-Llama-8B 포팅

기존 Llama-3-8B에선 모델은 MATH 정확도가 27% (FP16), 23% (DiffKV)로 둘 다 낮아 correctness 문제가 있었습니다. 디버깅을 더 쉽게 하기 위해 최소한의 수정으로 DeepSeek-R1-Distill-Llama-8B를 포팅하였습니다.
참고: paper에서는 FP16에서 28.1%, DiffKV에서 27.9%

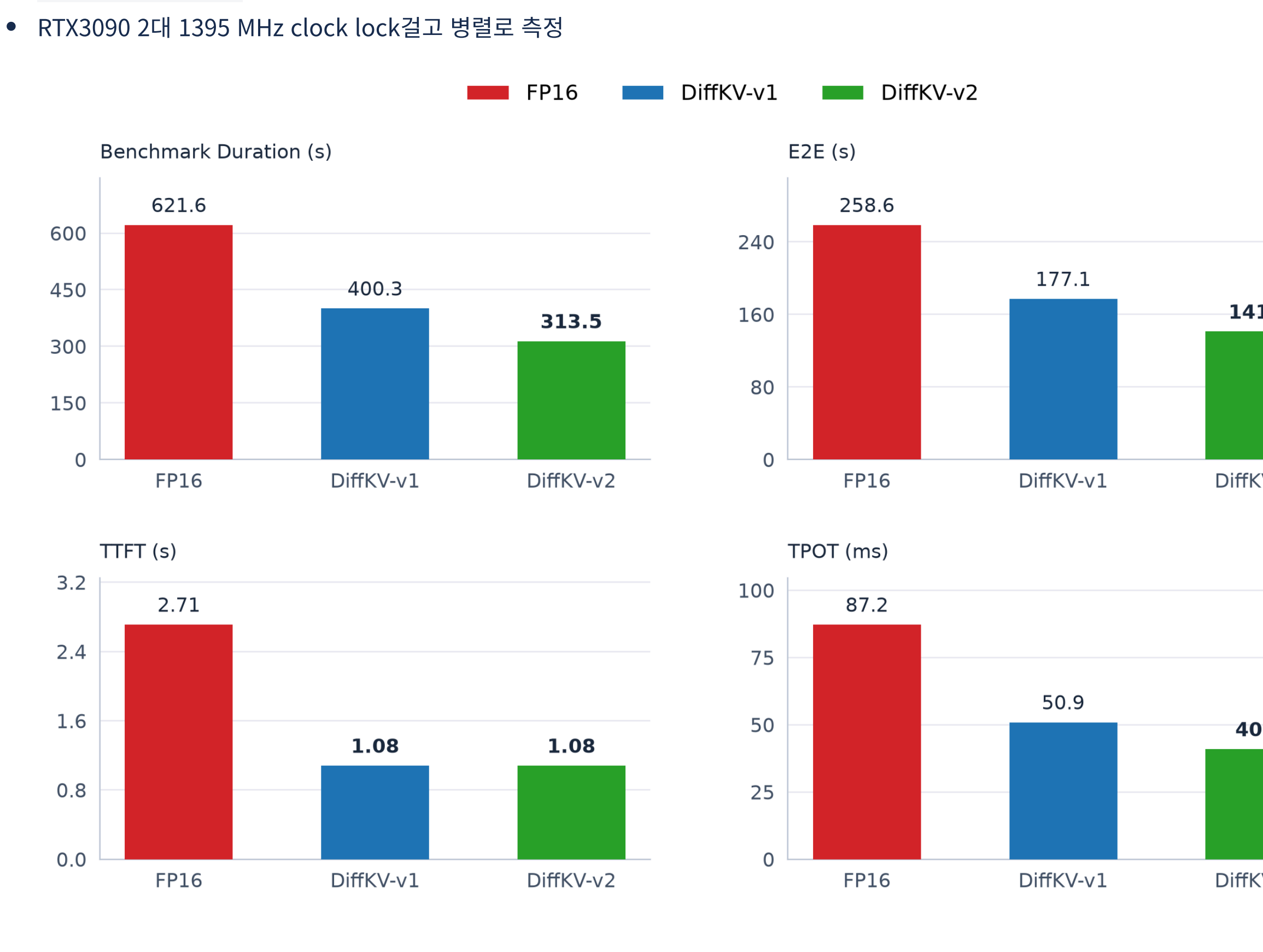
- Llama-3.1-8B 아키텍처 기반으로 layer 32, KV head 8, head dim 128, hidden, vocab 등 구성이 모두 같아 모든 GEMM, attention 커널, scheduler, KV management 등이 같은 경로로 실행됩니다. 따라서 R1에서 최적화한 효과는 기존 모델에도 동일하게 적용됩니다.
- 유일한 차이는 RoPE인데, cos, sin table만 R1에 호환되게 바꾸어주면 되고, 구 vLLM FP16 path (`0.2.6`)는 이를 지원하지 않으므로 이를 지원하도록 적절히 패치해줬습니다.

3.3. Correctness 측정 방법과 결과

- 입력: MATH-500 전체 dataset, prompt는 문제 + "Please reason step by step, and put your final answer within \boxed{}.", chat template 적용 후 token ID 고정
- 생성
  - temperature=0, top_p=1.0, 최대 8192 token
  - EOS에서 종료
  - 요청 500개를 한번에 동시 투입, scheduler가 max_num_seqs=64개씩 처리
  - engine 설정은 FP16과 DiffKV-v2 양쪽이 같음
    - max_model_len=9216, max_num_seqs=64, gpu_memory_utilization=0.9, eager execution
- 결과
  - FP16: `80.2% (401/500)`
  - DiffKV-v2: `77.2% (386/500)`
- 결과는 3%의 accuracy 차이로, 원인은 최대 생성 토큰 수에 출력이 짤려 오답처리가 되는 케이스일 것으로 추정되나, 별도로 확인이 필요함

3.4. E2E 측정 방법

- MATH-500 전체 500개 문제 중 50개만 측정
- Offline serving: engine에 request 50개를 한번에 투입 (burst)
- `ignore_eos=True`: EOS 종료 실행은 FP16 vs DiffKV가 OSL이 다르므로 공정한 비교를 위해서
- RTX3090 2대 1395 MHz clock lock걸고 병렬로 측정



	FP16	DiffKV-v1	DiffKV-v2
Benchmark duration (s)	621.6	400.3	**313.5**
Throughput (token/s)	280.0	434.8	**555.1 (1.98x)**
Mean E2E (s)	258.6	177.1	**141.0 (1.83x)**
Mean TTFT (s)	2.71	1.08	**1.08**
Mean TPOT (ms)	87.2	50.9	**40.9**
Mean decode batch	11.1	21.2	**21.2**
preemption 횟수	67	0	**0**
Peak KV usage	100%	70%	**70%**

- Peak KV usage = max{(전체 KV Cache block 수 - free block 수) / 전체 KV Cache block 수}

3.5. 주의사항

- 여기서 주의해야 할 점은, FP16의 TTFT가 DiffKV-v2 대비 2.5배 느린게 v1, v2 최적화의 결과로 해석하지 말아야 한다
- 구 vLLM은 prefill때 최대 ISL로 padding하는게 들어가 있어서 (_prepare_prompt) 고정 ISL인경우랑 서로 다른 ISL인 경우의 TPOT가 많이 달랐음
  - 고정 ISL로 offline serving: Prefill 시간이 DiffKV가 아주 살짝 더 느림
  - 서로 다른 ISL로 offline serving: Prefill 시간이 FP16 대비 2.5x 줄어듬
- 최신버전은 Varlen Flash Attention만 쓰기때문에, 이러한 패딩이 필요하지 않아 고정 ISL 케이스처럼 FP16 대비 성능이득이 없을 것임

4. 다음 주 계획

- 지금은 요청 50개를 한 번에 넣는 burst 방식으로만 E2E를 측정했는데, 그 외에도 다양한 옵션을 고려하여 실험을 진행해보기 (e.g., request rate를 조정해서 online serving 측정 (논문에서는 Poisson 도착))
- 현재 vLLM commit이 옛날거라, 최신 vLLM과 구현이 많이 다르고 (e.g., chunked prefill), 따라서 이걸로 만약 연구할 건덕지를 잡으려면 vLLM bringup을 해야함
  - vLLM version
    - DiffKV: https://github.com/vllm-project/vllm/commit/1db83e31a2468cae37f326a642c0a4c4edbb5e4f (2023/12/23)
    - 최신: https://github.com/vllm-project/vllm/releases/tag/v0.32.0 (2026/10/18)
  - kernel단과 serving단의 수정이 필요한데
    - 최신에서는 chunked prefill이 default이므로 이에 aware한 kernel을 새로 짜줘야하고
    - serving engine단은 그냥 아예 from scratch로 대략 몇천 몇만줄 정도를 짜야하는 로드가 요구됨
  - 이게 만만치 않은 작업이므로, 두 case 중 어디에 속하냐를 적은 사전실험 / 케이스 조사를 통해 진행할지를 결정해야함
    - i) 최신 연구 주제로 이어지기는 조금 힘들어서 이걸 그냥 기본적인 vLLM engine 도는 원리랑 실험, 최적화를 이해하는 "공부"용으로 쓸 것인가
    - ii) 정말 최신걸 성공적으로 끌어와서 SOTA의 병목을 파악하여 돌파해보는 실험을 진행할 것인가
  - hopefully ii)가 되면 정말로 좋겠지만, "된다"는 보장이 없으므로 적은 노력과 시간으로 아래를 확인해야함
    - i) 단일 Attention kernel이 DiffKV가 SOTA vLLM baseline대비 뚜렷한 성능 향상이 있는가?
    - ii) serving의 이득이 있는가? 그리고 이걸 어떻게 알 수 있나?

참고 자료

- https://dev.to/lewis_won/how-to-profile-and-optimise-kernels-2o48: Nsight system, Nsight compute를 이용한 profiling